Labb 7 – Admininloggning med GitHub och issues

Labb 7 av Inlämningsuppgift 2. Ges ut på L10, onsdag 11/11, och testar LR 4. Shoppens admin loggar in med sitt **GitHub-konto** via OAuth 2.0, och bara de GitHub-användare som du har utsett blir admin. Hela labben är VG-delen av uppgiften, och krävs inte för G. Admin kan också rapportera fel som issues i shoppens GitHub-repo, med shoppens egen GitHub-token. De gemensamma kraven i Inlämningsuppgift 2 gäller även här.

Det här får du

Kodgenomgången `github-oauth-walkthrough-start/` från L10 är en förebild för din shop. Den har ingen databas – i din shop sparas admin i databasen.

- **Färdigt:** inloggningen med GitHub – `state` (PKCE är bortkommenterat), kodbytet på servern, `GET /user`, behöriga admins i `.env` (`ADMIN_GITHUB_LOGINS`), shoppens JWT-cookie och utloggningen. Routes för issues finns också, skyddade med `requireAdmin`.
- **Inte färdigt – det bygger du själv:** `createIssue()` och `listOpenIssues()` i `githubAdapter.ts`, med länkar till GitHubs dokumentation ovanför varje funktion, och formuläret och listan i React.

Krav: inloggning med GitHub

- Skapa en **OAuth App** på GitHub under Settings → Developer settings. Client secret finns bara på servern.
- **Logga in med GitHub:** adminsidan har en länk som går till din server, som skickar vidare till GitHub. Efter inloggningen kommer användaren tillbaka till din callback-route.
- **state och/eller PKCE:** varje inloggning skyddas med `state`, PKCE eller båda. Med `state` kontrollerar servern i callbacken att värdet matchar det som sparades vid start. Med PKCE skickar servern en code verifier när koden byts. En inloggning som inte matchar avvisas.
- **Koden byts mot en token på servern**, med client secret, och med code verifier om du använder PKCE. Servern hämtar sedan användaren från GitHubs API. GitHubs svar valideras, t.ex. med Zod.

- **Minsta scope:** be bara om `read:user`.
- **Bara admins:** en lista med GitHub-användarnamn i `.env` avgör vem som är admin. Andra GitHub-användare får ett begripligt fel.
- **Shoppens egen inloggning:** efter callbacken loggar shoppen in admin med en session eller en JWT i en httpOnly-cookie. GitHubs token når aldrig webbläsaren. Motivera ditt val i README.
- **Adminroutes är skyddade** på servern, t.ex. orderlistan från Inlämningsuppgift 1. Utan inloggning svarar de 401.
- Admin kan logga ut.
- Admin sparas i databasen, t.ex. med GitHubs `id`. Använd `id` och inte användarnamnet, som kan ändras.
- Avbruten inloggning och fel från GitHub ger ett begripligt meddelande i stället för en kraschad sida.

Krav: issues

- **Shoppens egen token:** skapa en fine-grained personal access token på GitHub med behörigheten **Issues: Read and write**, bara för shoppens repo och med ett utgångsdatum. Token finns bara i `.env` på servern och når aldrig webbläsaren.
- **Rapportera fel:** adminsidan har ett formulär med rubrik och beskrivning. Servern skapar en issue i shoppens repo via GitHubs REST-API, med shoppens token. GitHub visar shoppens konto som författare, så issuen visar i texten vilken admin som rapporterade den.
- **Öppna issues:** adminsidan listar repots öppna issues, med länk till varje issue på GitHub.
- **Skyddat:** routes för issues kräver admininloggning och svarar 401 utan den.
- Fel från GitHub, t.ex. en ogiltig token eller fel repo, ger ett begripligt meddelande i stället för en kraschad sida.

Den deployade shoppen

Inloggningen och issues ska fungera i den deployade shoppen. Det är där de testas.

- Lägg till den deployade shoppens callback-adress bland OAuth Appens **Redirect URIs**, t.ex. `https://din-shop.vercel.app/api/auth/github/callback`. Adressen måste stämma exakt, annars avvisar GitHub inloggningen. Låt **Allow wildcard**

matching vara avbockad.

- Lägg nycklarna och callback-adressen i plattformens miljövariabler, t.ex. i Vercel eller Render. Även shoppens GitHub-token.
- Säkrare: skapa en **egen OAuth App för produktion**, så att dess client secret aldrig finns på din dator.
- Shoppens cookie ska vara `secure`.
- Vill du även testa lokalt? Lägg till `http://localhost:5173/api/auth/github/callback` som en andra adress, och lägg nycklarna i `.env`. En OAuth App kan ha upp till 10 redirect URIs.

Klart när

Inloggning: i den deployade shoppen loggar en admin in med GitHub och kommer tillbaka inloggad till adminsidan. GitHub frågar bara om läsrättighet till profilen. En GitHub-användare som inte är admin, och en avbruten inloggning, får ett begripligt fel. Adminroutes svarar 401 utan inloggning och fungerar efter inloggning. Utloggning fungerar. Ingen client secret eller GitHub-token finns i webbläsaren. README motiverar valet av session eller JWT.

Issues: admin skapar en issue från formuläret på adminsidan, och den syns i repot med adminens användarnamn i texten. Listan visar repots öppna issues med länkar till GitHub. Routes för issues svarar 401 utan inloggning. Shoppens GitHub-token finns varken i webbläsaren eller i koden på GitHub.